

UNIT-4

Array in c

An array is defined as the collection of similar type of data items stored at contiguous memory locations.

Arrays are the derived data type in C programming language which can store the primitive type of data such as int, char, double, float, etc.

It also has the capability to store the collection of derived data types, such as pointers, structure, etc.

The array is the simplest data structure where each data element can be randomly accessed by using its index number.

C array is beneficial if you have to store similar elements.

For example, if we want to store the marks of a student in 6 subjects, then we don't need to define different variables for the marks in the different subject.

Instead of that, we can define an array which can store the marks in each subject at the contiguous memory locations.

By using the array, we can access the elements easily. Only a few lines of code are required to access the elements of the array.

Properties of Array

The array contains the following properties.

Exception Handling in Java

Each element of an array is of same data type and carries the same size, i.e., int = 4 bytes.

Elements of the array are stored at contiguous memory locations where the first element is stored at the smallest memory location.

Elements of the array can be randomly accessed since we can calculate the address of each element of the array with the given base address and the size of the data element.

Advantage of C Array

- 1) Code Optimization:** Less code to access the data.
- 2) Ease of traversing:** By using the for loop, we can retrieve the elements of an array easily.
- 3) Ease of sorting:** To sort the elements of the array, we need a few lines of code only.
- 4) Random Access:** We can access any element randomly using the array.

Disadvantage of C Array

- 1) Fixed Size:** Whatever size, we define at the time of declaration of the array, we can't exceed the limit. So, it doesn't grow the size dynamically.

Types of C Arrays:

There are 2 types of C arrays. They are,

1. **One dimensional array**
2. **Multi-dimensional array** (Two-dimensional array, Three-dimensional array, Four-dimensional array, etc...)

One Dimensional Array in C:

One dimensional array is an array that has **only one subscript specification** that is needed to specify a particular element of an array.

A one-dimensional array is a structured collection of components (often called array elements) that can be accessed individually by specifying the position of a component with a single index value.

Syntax: `data-type arr_name[array_size];`

Prepared By: Pooja Sasane

Programming in C

Rules for Declaring One Dimensional Array

1. An array variable must be declared before being used in a program.
2. The declaration must have a data type(int, float, char, double, etc.), variable name, and subscript.
3. The subscript represents the size of the array. If the size is declared as 10, programmers can store 10 elements.
4. An array index always starts from 0. For example, if an array variable is declared as `s[10]`, then it ranges from 0 to 9.
5. Each array element is stored in a separate memory location.

Declaration of C Array

We can declare an array in the c language in the following way.

Syntax:

1. `data_type array_name[array_size];`

Now, let us see the example to declare the array.

Method of declaring an array

For Example:

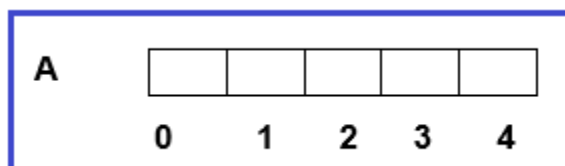
If you want an integer-type array, let's say `int A[5];`

Then, you will get 5 integers, and the array name is "A" and you can access all those integers using the name "A".

So, "A" is an array and you get 5 integers spaces and the indices will be 0,1,2,3,4.

So, 5 locations mean indices will start from 0 and end at 4, a total of 5 spaces.

For a better understanding, please have a look at the below image.



Now we can store 5 integers. Every location is an integer. If we assume an integer is taking 2 bytes then there are a total of 10 bytes i.e. 2 bytes each.

Now each location can be accessed like `A[0]=27;` //Here, 27 will store in array "A" with indices 0.

Now if I store `A[1]=10;` //10 is stored in a place with indices 1.

For a better understanding, please have a look at the below image,

A	27	10			
	0	1	2	3	4

Array declaration by initializing elements:

```
int arr[] = { 10, 20, 30, 40 }
```

Here, the compiler creates an array of size 4. The above statement is the same as the below statement.

```
int arr[4] = {10, 20, 30, 40}
```

Array declaration by specifying the size and initializing elements:

```
int arr[6] = { 10, 20, 30, 40 }
```

Here, the compiler creates an array of size 6, initializes the first 4 elements as specified by the user, and the rest two elements as 0. The above statement is the same as the below statement.

```
int arr[] = {10, 20, 30, 40, 0, 0}
```

Element	2	3	15	8	48	13
Index	0	1	2	3	4	5

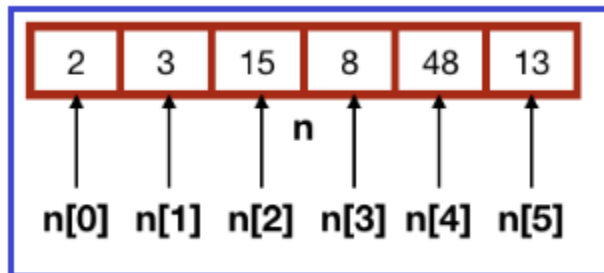
0, 1, 2, 3, 4, and 5 are indices. It is like they are the identity of 6 different elements of an array. The index always starts at 0. So, the first element of an array has an index of 0.

Note: The index of an array starts with 0. We access any element of an array using its index:

Syntax: `array_name[index]`

For example:

if the name of an array is 'n', then to access the first element (which is at 0 indexes), we write `n[0]`.



Here,

`n[0]` is 2
`n[1]` is 3
`n[2]` is 15
`n[3]` is 8
`n[4]` is 48
`n[5]` is 13

Note: `n[0]`, `n[1]`, etc. are like any other variables we were using till now i.e., we can set there value as `n[0] = 5`; like we do with any other variables (`x = 5`;; `y = 6`;; etc.).

Assigning Values to Array in C:

By writing `int n[] = { 2,4,8 };`, we are declaring and assigning values to the array at the same time, thus initializing it.

But when we declare an array like `int n[3];`, we need to assign values to it separately.

Because '`int n[3];`' will allocate space for 3 integers in memory but there are no integers in that space.

To initialize it, assign a value to each of the elements of the array as shown below.

```
n[0] = 2;  
n[1] = 4;  
n[2] = 8;
```

It is just like we are declaring some variables and then assigning values to them.

```
int x,y,z;  
x=2;  
y=4;  
z=8;
```

Thus, the first way of assigning values to the elements of an array is by doing so at the time of its declaration i.e.

`int n[]={ 2,4,8 };` And the second method is declaring the array first and then assigning values to its elements as shown below.

```
int n[3]; //Declaring an array  
  
n[0] = 2;  
n[1] = 4; //Initializing array  
n[2] = 8;
```

We can initialize each element of the array by using the index.

Consider the following example.

1. `marks[0]=80; //initialization of array`
2. `marks[1]=60;`
3. `marks[2]=70;`
4. `marks[3]=85;`
5. `marks[4]=75;`

80	60	70	85	75
----	----	----	----	----

marks[0] marks[1] marks[2] marks[3] marks[4]

Initialization of Array

How to access an array?

We can access all the elements of an array one by one using a “for” loop. To understand this better please have a look at the following code.

```
int main()
{
int A[5]={2, 4,6,8,10};
int i;
for(i=0;i<5;i++)
{
printf("%d",A[i]);
}
}
```

You can understand this by treating n[0], n[1], and n[2] as similar to the different variables you used before.

Just like a variable, an array can be of any other data type also.

Array declaration, initialization, and accessing

Array declaration syntax: **data_type arr_name [arr_size];**

Array initialization syntax: **data_type arr_name [arr_size]=(value1, value2, value3,...);**

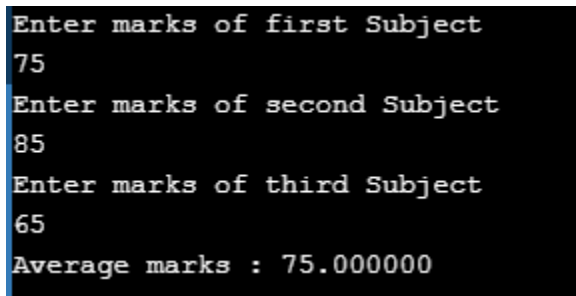
Array accessing syntax: **arr_name[index];**

float f[] = { 1.1, 1.4, 1.5}; Here, 'f' is an array of floats.

Program: Array Example in C Language

```
#include <stdio.h>
#include <conio.h>
void main()
{
    int marks[3];
    float average;
    printf("Enter marks of first student\n");
    scanf(" %d", &marks[0]);
    printf("Enter marks of second student\n");
    scanf(" %d", &marks[1]);
    printf("Enter marks of third student\n");
    scanf(" %d", &marks[2]);
    average = (marks[0] + marks[1] + marks[2]) / 3.0;
    printf("Average marks : %f\n", average);
    getch();
}
```

Output:



```
Enter marks of first Subject
75
Enter marks of second Subject
85
Enter marks of third Subject
65
Average marks : 75.000000
```

Runtime Array initialization in C:

An array can also be initialized at runtime using scanf() function. This approach is usually used for initializing large arrays or to initialize arrays with user-specified values.

Runtime Array initialization Examples in C:

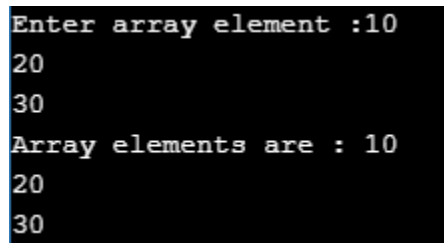

```

#include<stdio.h>
#include<conio.h>

void main()
{
int arr[3];
int i, j;
printf("Enter array element :");
for(i = 0; i < 3; i++)
{
scanf("%d", &arr[i]); //Run time array initialization
}
printf("Array elements are : ");
for(j = 0; j < 3; j++)
{
printf("%d\n", arr[j]);
}
getch();
}

```

Output:



```

Enter array element :10
20
30
Array elements are : 10
20
30

```

Input data into the array in C Language

Here, the size of the array is 5. Inside the loop, we are displaying a message to the user to enter the number.

All the input values are stored in similar array elements using the scanf function.

```

for (i=0; i<5; i++)
{
    printf("Enter the number %d \n", (i+1));
    scanf("%d", &num[i]);
}

```

Prepared By: Pooja Sasane

Programming in C

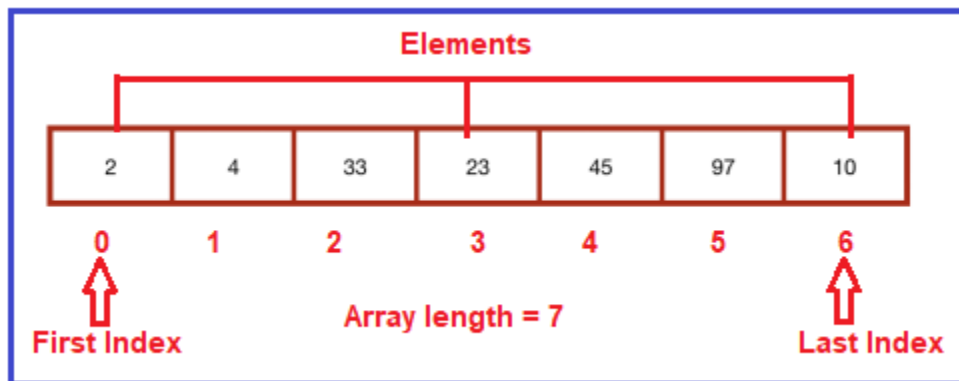
Reading out data from an array

If we want to display the elements of the array then we can display like this.

```
for (i=0; i<5;i++)  
{  
    printf("num[%d]\n", num[i]);  
}
```

Access Array Elements in C Language:

We can access elements of an array by index. Suppose we declared an array n. The first element is n[0], the second element is n[1], and so on.



Arrays have 0 as the first index, not 1.

For example, n[0] is the first element.

If the size of an array is 7, to access the last element, the 7-1 index is used. For example n[6]=10

Note: Array allocates contiguous memory.

Thus, if the address of the first element of an array of integers is 223698684 then the address of the second element will be 223698688 (223698684+4 (4 is the size of 1 integer)) and the third will be 223698692, and so on.

This means that the memories of all elements of an array are allocated together in a continuous manner.

Program:

```
#include<stdio.h>
#include<conio.h>
void main()
{
int n[5]={0, 1, 2, 3, 4};
printf("%d", n[0]);
printf("%d", n[1]);
printf("%d", n[2]);
printf("%d", n[3]);
printf("%d", n[4]);
getch();
}
```

Output: 0 1 2 3 4

Example: Integer array example:

```
int age [5];
int age[5]={0, 1, 2, 3, 4};

age[0]; /*0 is accessed*/
age[1]; /*1 is accessed*/
age[2]; /*2 is accessed*/
```

Example: Character array example:

```
char str[10];
char str[10]={'H','a','i'};
(or)
char str[0] = 'H';
char str[1] = 'a';
char str[2] = 'i';

str[0]; /*H is accessed*/
str[1]; /*a is accessed*/
str[2]; /*i is accessed*/
```

Example of One Dimensional Array in C

```
#include<stdio.h>
#include<conio.h>
void main()
{
    int i;
    int arr[5] = {10,20,30,40,50};
    // declaring and Initializing array in C
    //To initialize all array elements to 0, use int arr[5]={0};
    /* Above array can be initialized as below also
    arr[0] = 10;
    arr[1] = 20;
    arr[2] = 30;
    arr[3] = 40;
    arr[4] = 50; */
    for (i=0;i<5;i++)
    {
        // Accessing each variable
        printf("value of arr[%d] is %d \n", i, arr[i]);
    }
    getch();
}
```

Output:

```
value of arr[0] is 10
value of arr[1] is 20
value of arr[2] is 30
value of arr[3] is 40
value of arr[4] is 50
```

Example:

Create an integer array with size 5 and then calculate the larger element of that array using the function.

```
#include<stdio.h>
#include<conio.h>
```

Prepared By: Pooja Sasane

Programming in C

```

void main()
{
int arr[5];
int max, i;
max = arr[0];
printf("Enter the array elements : ");
for(i = 0; i<5; i++)
scanf("%d", &arr[i]);
    for(i = 1; i<5; i++)
    {
        if (arr[i] > max)
            max = arr[i];
    }
printf("The largest element is : %d", max);
getch();
}

```

Output:

```

Enter the array elements : 55 77 82 68 60
The largest element is : 82

```

C Array Example: Sorting an array

In the following program, to sort the array in decending order.

```

#include<stdio.h>
#include<conio.h>
void main ()
{
    int i, j,temp;
    int a[10] = { 10, 9, 7, 101, 23, 44, 12, 78, 34, 23};
    for(i = 0; i<10; i++)
    {
        for(j = i+1; j<10; j++)
        {
            if(a[j] > a[i])

```

```

        {
            temp = a[i];
            a[i] = a[j];
            a[j] = temp;
        }
    }
}
printf("Printing Sorted Element List ...\n");
for(i = 0; i < 10; i++)
{
    printf("%d\n", a[i]);
}
getch();
}

```

Two-Dimensional Array in C Language:

The most popular and commonly used multi-dimensional array is a two-dimensional array. The 2-D arrays are used to store data in the form of a table. We also use 2-D arrays to create mathematical matrices.

The Two-dimensional array is nothing but a table with rows and columns. A two-dimensional array can be expressed as a contiguous and tabular block in memory where the data can be stored in a table structure.

Declaration of Two-Dimensional Array in C Language

Syntax: `datatype arrayName [ rowSize ] [ columnSize ];`

Initialization of 2D Array in C

In the 1D array, we don't need to specify the size of the array if the declaration and initialization are being done simultaneously.

However, this will not work with 2D arrays.

We will have to define at least the second dimension of the array.

The two-dimensional array can be declared and defined in the following way.

Syntax:

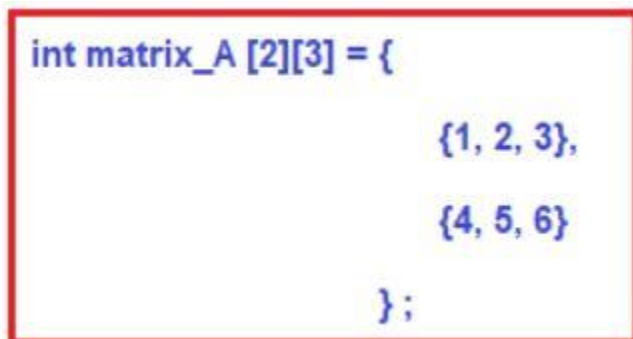
```
datatype arrayName [rows][colms] = {{r1c1value, r1c2value, ...},{r2c1, r2c2,...}...};
```

Example:

```
int matrix_A [2][3] = { {1, 2, 3},{4, 5, 6} };
```

The above declaration of two-dimensional array reserves 6 contiguous memory locations of 2 bytes each in the form of 2 rows and 3 columns.

And the first row is initialized with values 1, 2 & 3 and the second row is initialized with values 4, 5 & 6. We can also initialize as follows:



```
int matrix_A [2][3] = {  
    {1, 2, 3},  
    {4, 5, 6}  
};
```

Example

```
int a[3][3];
```

Once we declare the 2D Array, it will look like as shown in the picture below:

	Column	Column	Column
	0	1	2
Row 0	0, 0	0, 1	0, 2
Row 1	1, 0	1, 1	1, 2
Row 2	2, 0	2, 1	2, 2

In the above image, you can see that we have created a 2D Array having 3 rows and 3 columns.

We can call the above array as a 3x3 Matrix.

The first row and column always start with index 0.

Note: A 2D array is used to store data in the form of a table.

Example 1: (with row and column size mentioned)

`int a[3][3]= {{1,2,3}, {4,5,6}, {7,8,9}};`

	0	1	2
0	1	2	3
1	4	5	6
2	7	8	9

Points to Remember while working with 2D Array in C:

1. In a 2d array, elements are arranged in rows and columns format.
2. When we are working with the 2D array we require to use 2 subscript operators which indicate the row and column sizes.
3. In a 2D array when we are referring to one subscript operator then it gives the rows address, 2nd subscript operator gives the element
4. The main memory of the 2D array is rows and the sub-memory is columns.

5. On the 2D array, the array name always gives the main memory that is the 1st-row base address, arr+1 will give the next row base address.

Two-dimensional array example in C

```
#include<stdio.h>
#include<conio.h>
void main(){
int i=0,j=0;
int arr[4][3]={{1,2,3},{2,3,4},{3,4,5},{4,5,6}};
//traversing 2D array
for(i=0;i<4;i++)
{
    for(j=0;j<3;j++)
    {
        printf("arr[%d] [%d] = %d \n",i,j,arr[i][j]);
    } //end of j
} //end of i
getch();
}
```

Output

```
arr[0][0] = 1
arr[0][1] = 2
arr[0][2] = 3
arr[1][0] = 2
arr[1][1] = 3
arr[1][2] = 4
arr[2][0] = 3
arr[2][1] = 4
arr[2][2] = 5
arr[3][0] = 4
arr[3][1] = 5
arr[3][2] = 6
```

C 2D array example: Storing elements in a matrix and printing it.

```
#include <stdio.h>
#include <conio.h>
void main ()
{
    int arr[3][3],i,j;
    for (i=0;i<3;i++)
    {
        for (j=0;j<3;j++)
        {
            printf("Enter a[%d][%d]: ",i,j);
            scanf("%d",&arr[i][j]);
        }
    }
    printf("\n printing the elements ....\n");
    for(i=0;i<3;i++)
    {
        printf("\n");
        for (j=0;j<3;j++)
        {
            printf("%d\t",arr[i][j]);
        }
    }
    getch();
}
```

Output

```
Enter a[0][0]: 56
Enter a[0][1]: 10
Enter a[0][2]: 30
Enter a[1][0]: 34
Enter a[1][1]: 21
Enter a[1][2]: 34

Enter a[2][0]: 45
Enter a[2][1]: 56
Enter a[2][2]: 78
```

```
printing the elements ....
```

```
56      10      30
34      21      34
45      56      78
```

Example 2

Program to input numbers in a 3x3 Matrix and print only even numbers if present in the matrix.

```
#include <stdio.h>
#include <conio.h>

void main()
{
    int a[3][3], r,c;

    printf("Enter 9 numbers\n");
    for(r=0; r<3; r++)
    {
        for(c=0; c<3; c++)
        {
            scanf("%d",&a[r][c]);
        }
    }

    printf("\nEven Numbers\n");
    for(r=0; r<3; r++)
    {
        for(c=0; c<3; c++)
        {
            if(a[r][c]%2==0)
```

```

        printf("%d ",a[r][c]);
    }
}
}
getch();
}

```

Output

Enter 9 numbers

84

36

41

98

34

57

20

31

24

Even Numbers

84 36 98 34 20 24

Here, you can see that we have entered numbers in a 3x3 matrix using nested for loop.

After that we have run another nested for loop to fetch the numbers from the 2D array and print only those numbers which are divisible by 2.

Passing Array to Function in C

In C, there are various general problems which requires passing more than one variable of the same type to a function.

For example, consider a function which sorts the 10 elements in ascending order.

Such a function requires 10 numbers to be passed as the actual parameters from the main function.

Here, instead of declaring 10 different numbers and then passing into the function, we can declare and initialize an array and pass that into the function.

This will resolve all the complexity since the function will now work for any number of values.

As we know that the array_name contains the address of the first element.

Here, we must notice that we need to pass only the name of the array in the function which is intended to accept an array.

The array defined as the formal parameter will automatically refer to the array specified by the array name defined as an actual parameter.

Consider the following syntax to pass an array to the function.

1. `functionname(arrayname);` *//passing array*

Methods to declare a function that receives an array as an argument

There are 3 ways to declare the function which is intended to receive an array as an argument.

First way:

1. `return_type function(type arrayname[])`

Declaring blank subscript notation [] is the widely used technique.

Second way:

1. `return_type function(type arrayname[SIZE])`

Optionally, we can define size in subscript notation [].

C language passing an array to function example

```
#include<stdio.h>
#include<conio.h>
int minarray(int arr[],int size)
{
    int min=arr[0];
    int i=0;
    for(i=1;i<size;i++)
    {
        if(min>arr[i]){
            min=arr[i];
        }
    }
    //end of for
    return min;
}
//end of function

void main()
{
    int i=0,min=0;
    int numbers[]={4,5,7,3,8,9};//declaration of array

    min=minarray(numbers,6);//passing array with size
    printf("minimum number is %d \n",min);
    getch();
}
```

Output

```
minimum number is 3
```

C function to sort the array

```

#include<stdio.h>
#include<conio.h>
void Sort(int[]);
void main ()
{
    int arr[10] = { 10, 9, 7, 101, 23, 44, 12, 78, 34, 23};
    Sort(arr);
    getch();
}
void Sort(int a[]) //array a[] points to arr.
{
    int i, j, temp;
    for(i = 0; i<10; i++)
    {
        for(j = i+1; j<10; j++)
        {
            if(a[j] < a[i])
            {
                temp = a[i];
                a[i] = a[j];
                a[j] = temp;
            }
        }
    }
    printf("Printing Sorted Element List ...\n");
    for(i = 0; i<10; i++)
    {
        printf("%d\n",a[i]);
    }
}

```

Output

```

Printing Sorted Element List ...
7
9

```

```
10
12
23
23
34
44
78
101
```

Some Important programs in c

Fibonacci Series in C

Fibonacci Series in C: In case of fibonacci series, *next number is the sum of previous two numbers* for example 0, 1, 1, 2, 3, 5, 8, 13, 21 etc. The first two numbers of fibonacci series are 0 and 1.

```
#include<stdio.h>
#include<conio.h>
void main()
{
    int n1=0,n2=1,n3,i,number;
    printf("Enter the number of elements:");
    scanf("%d",&number);
    printf("\n%d %d",n1,n2);//printing 0 and 1
    for(i=2;i<number;++i)//loop starts from 2 because 0 and 1 are already printed
    {
        n3=n1+n2;
        printf(" %d",n3);
        n1=n2;
        n2=n3;
    }
    getch();
}
```



```
}
```

Output:

Enter the number of elements:15

0 1 1 2 3 5 8 13 21 34 55 89 144 233 377

Prime Number program in C

Prime number is a number that is greater than 1 and divided by 1 or itself.

In other words, prime numbers can't be divided by other numbers than itself or 1.

For example 2, 3, 5, 7, 11, 13, 17, 19, 23.... are the prime numbers.

```
#include<stdio.h>
```

```
#include<conio.h>
```

```
void main()
```

```
{
```

```
int n,i,m=0,flag=0;
```

```
printf("Enter the number to check prime:");
```

```
scanf("%d",&n);
```

```
m=n/2;
```

```
for(i=2;i<=m;i++)
```

```
{
```

```
if(n%i==0)
```

```
{
```

```
printf("Number is not prime");
```

```
flag=1;
```

```
break;
```

```
}
```

```
}
```

```
if(flag==0)
```

```
printf("Number is prime");
```

```
getch();
```

```
}
```

Output:

```
Enter the number to check prime:56
Number is not prime

Enter the number to check prime:23
Number is prime
```

Palindrome program in C

Palindrome number in c: A **palindrome number** is a number that is same after reverse.
For example 121, 34543, 343, 131, 48984 are the palindrome numbers.

```
#include<stdio.h>
```

```
#include<conio.h>
```

```
void main()
```

```
{
```

```
int n,r,sum=0,temp;
```

```
printf("enter the number=");
```

```
scanf("%d",&n);
```

```
temp=n;
```

```
while(n>0)
```

```
{
```

```
r=n%10;
```

```
sum=(sum*10)+r;
```

```
n=n/10;
```

```
}
```

```
if(temp==sum)
```

```
printf("palindrome number ");
```

```
else
```

```
printf("not palindrome");
```

```
getch();
```

```
}
```

Output:

```
enter the number=151
palindrome number

enter the number=5621
not palindrome number
```

Factorial Program in C

Factorial Program in C: Factorial of n is the *product of all positive descending integers*. Factorial of n is denoted by $n!$. For example:

1. $5! = 5*4*3*2*1 = 120$
2. $3! = 3*2*1 = 6$

```
#include<stdio.h>
#include<conio.h>
```

```
void main()
{
    int i,fact=1,number;
    printf("Enter a number: ");
    scanf("%d",&number);
    for(i=1;i<=number;i++){
        fact=fact*i;
    }
    printf("Factorial of %d is: %d",number,fact);
    getch();
}
```

Output:

```
Enter a number: 5
```

Factorial of 5 is: 120

Armstrong Number in C

Armstrong number is a number that is equal to the sum of cubes of its digits. For example 0, 1, 153, 370, 371 and 407 are the Armstrong numbers.

Let's try to understand why **153** is an Armstrong number.

1. $153 = (1*1*1) + (5*5*5) + (3*3*3)$
2. where:
3. $(1*1*1) = 1$
4. $(5*5*5) = 125$
5. $(3*3*3) = 27$
6. So:
7. $1 + 125 + 27 = 153$

the c program to check Armstrong Number in C.

```
#include<stdio.h>
#include<conio.h>
```

```
void main()
{
    int n,r,sum=0,temp;
    printf("enter the number=");
    scanf("%d",&n);
    temp=n;
    while(n>0)
    {
        r=n%10;
        sum=sum+(r*r*r);
    }
```

```

n=n/10;
}
if(temp==sum)
printf("armstrong number ");
else
printf("not armstrong number");
getch();
}

```

Output:

```

enter the number=153
armstrong number

enter the number=5
not armstrong number

```

C Program to swap two numbers without third variable

We can swap two numbers without using third variable. There are two common ways to swap two numbers without using third variable:

a simple c example to swap two numbers without using third variable.

1. `#include<stdio.h>`
2. `#include <conio.h>`
3. **Void** main()
4. {
5. **int** a=10, b=20;
6. printf("Before swap a=%d b=%d",a,b);
7. `a=a+b; //a=30 (10+20)`
8. `b=a-b; //b=10 (30-20)`
9. `a=a-b; //a=20 (30-10)`
10. printf("\nAfter swap a=%d b=%d",a,b);
11. **getch();**
12. }

Output:

Prepared By: Pooja Sasane

Programming in C

```
Before swap a=10 b=20  
After swap a=20 b=10
```